

Generative AI - Assignment 2

Submission Sheet

Name	Darrsheni Sapovadia
Student ID	27PGAI0063
Submitted on	19 September 2026
Parts completed	Part 1 (60 marks) and Part 2 (40 marks)

GitHub repository

<https://github.com/darrshenisapovadia/genai>

The repository is public and has one folder per assignment. Everything for this one is in the Assignment 2 folder. The notebook is committed with its outputs still in it, so the results can be read straight off GitHub without running anything.

What is in the Assignment 2 folder

File	What it is
Generative AI - Assignment 2.ipynb	Part 1 and Part 2, run end to end with outputs saved
mutual_funds_data.csv	The Kaggle mutual funds dataset, committed so the notebook runs after a plain clone
requirements.txt	The packages needed, with LangChain pinned to 0.3
.gitignore	Keeps the .env file and the Chroma database out of the repo

How to run it

1. git clone <https://github.com/darrshenisapovadia/genai>
2. Go into the Assignment 2 folder and run pip install -r requirements.txt
3. Install Ollama and run ollama pull nomic-embed-text
4. Create a .env file in the same folder with GROQ_API_KEY=<your key>
5. Open the notebook in Jupyter and run all cells

The notebook reads the CSV with a relative path, so it has to be run from inside the Assignment 2 folder. If you run it a second time, delete the chroma_langchain_db folder first, otherwise the same 1,000 funds get added to the collection again.

Part 1: RAG pipeline and Gradio app

The CSV has 35,559 rows. CSVLoader reads it and each row becomes a Document holding the scheme name, fund house, scheme type, scheme category, NAV and date, with scheme type and category also kept in the metadata. The first 1,000 are embedded with nomic-embed-text through Ollama and stored in Chroma with a persist directory, each with its own UUID. The retriever is a plain similarity search that brings back the top 5 funds.

The output is two Pydantic models: MutualFund (scheme name, fund house, scheme type and category) and MutualFundOutput (the reasoning plus a list of schemes). PydanticOutputParser turns the model's reply into those objects. The prompt takes the user's preferences, the retrieved funds and the format instructions, and it tells the model that no two picks can share both the same fund house and the same scheme type.

GPT-OSS-120B was already in the starter code. The two models I added are GPT-OSS-20B and Qwen3.8-27B, both on Groq, so one API key covers all three. They sit in the dropdown under LLM Settings (Advanced) along with the temperature slider, which goes from 0 to 2.

For the manual test I used "I want a low risk investment portfolio". It came back with three debt funds from Sundaram, Franklin Templeton and LIC, which is a sensible answer for someone who wants low risk. I also tried the example from the brief, "high return portfolio, only from Aditya Birla fund house", in the Gradio app with all three models. Each one stuck to Aditya Birla funds and picked schemes of different types.

Part 2: LangGraph

The same pipeline rebuilt as a graph. The State holds the preferences, the retrieved documents and the parsed answer. The retrieve node only returns the context and the generate node only returns the answer, and the graph runs START, retrieve, generate, END. The notebook draws the graph as well.

With the same low risk question the graph picked the same three funds as Part 1, which was a good check that both versions are doing the same thing.

Things I had to work around

The starter code says to use llama3.2:1b for the embeddings. On my version of Ollama (0.34) that model can't produce embeddings at all and the server returns "This server does not support embeddings". llama3.2 is a chat model, so I switched to nomic-embed-text, which is the embedding model named in the problem statement. Retrieval works well with it. The Aditya Birla question brings back only Aditya Birla funds.

Sending all 1,000 documents to Ollama in one go failed every so often with a "connection refused" error part way through, even though Ollama itself was still running. Adding them in batches of 100 fixed it, and three full runs in a row went through without an error.

Llama-3.3-70B was my first choice for the third model, but Groq has since removed it, so I went with Qwen3.8-27B. On the free tier Qwen is limited to 1,000 output tokens a minute, and by default the request asks for more than that, so Groq rejected it before it even ran. Capping max_tokens at 900 for Qwen in get_llm fixed it, but two Qwen requests in the same minute can still hit the limit.

The starter imports (langchain.prompts, langchain.output_parsers and langchain.schema) only exist up to LangChain 0.3 and were removed in 1.x, which is why requirements.txt pins the 0.3 versions.